

我的 OpenClaw Token 账单降了72%，只因装了这个插件

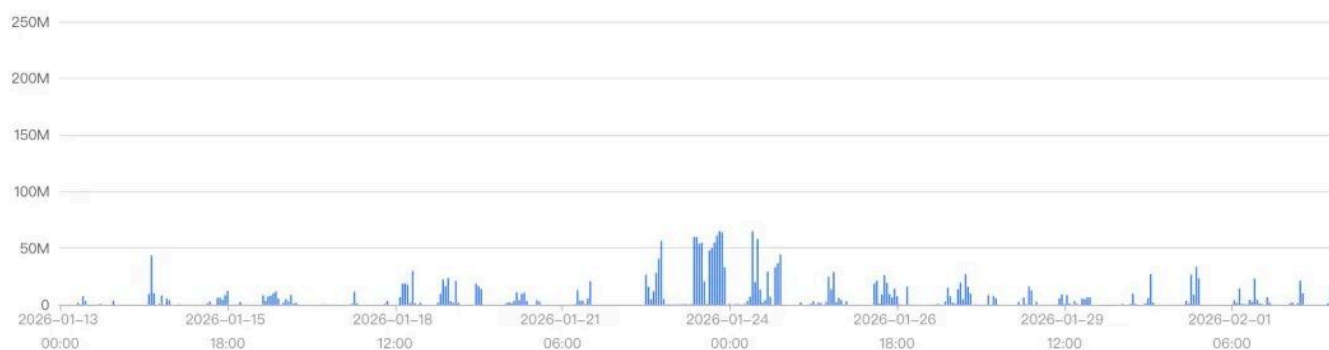
mp.weixin.qq.com/s/Fh-LmYDnm7O5IE3eibgUJQ

林月半子聊AI

Original

最近 OpenClaw 玩法很多，组建 AI 打工天团，养成系 AI 女友，每一个都让人上头。

但激情过后，打开 API 账单，瞬间透心凉。



这种吞金速度，别说跑应用了，多聊几句都心疼。

为什么会变成"吞金兽"？

因为在大模型领域，Context 即成本。

OpenClaw 默认把所有对话，全塞进上下文，Agent 带着一堆无关信息，Token 消耗指数级增长。更糟的是，噪点太多，模型注意力分散，甚至出现幻觉。

而且我发现一个普遍现象，群里不少朋友用 OpenClaw，都是一个主 Agent 干所有事，写代码、头脑风暴、写文章、查资料.....时间一长，记忆文件越堆越多，每次执行任务，都要读一大坨历史。

结果就是，Agent 开始"神经错乱"。

你让它写代码，它突然蹦出上周的公众号大纲；你让它分析需求，它把调试 Bug 的记忆也塞进来了。不同任务的上下文交叉污染，精准度直线下降，Token 还在疯狂燃烧。

我在X上刷到一个本地方案，然后放弃了

看到这里，硬核玩家可能会说，"自己搭个本地向量库不就行了？"

我也想过。

前阵子在 X 上刷到有人分享了一个叫 QMD 的方案，看起来挺美好，能让 OpenClaw 拥有本地检索记忆。

但打开文档，看到这段话，可能会倒吸一口凉气：

QMD - Query Markup Documents

An on-device search engine for everything you need to remember. Index your markdown notes, meeting transcripts, documentation, and knowledge bases. Search with keywords or natural language. Ideal for your agentic flows.

QMD combines BM25 full-text search, vector semantic search, and LLM re-ranking—all running locally via node-llama-cpp with GGUF models.

这意味着什么？

你得先交一笔"设备税"：

第一，下载模型，首次运行会自动拉取约 2G 的 GGUF 模型到本地。

第二，常驻进程，后台必须跑一个 node-llama-cpp，只要 Agent 在用，就不能关。

第三，资源爆炸，向量化和重排序，都是 CPU 密集型任务。

实际体验就是，你唤醒 Agent 准备干活，笔记本风扇开始狂转，CPU 占用飙升。

还没写出第一行代码，16G内存已经吃掉一大半。

为了省 API 费，把生产力工具搞成了卡顿模拟器。

不值得。

MemOS：把"大脑皮层"搬到云端

然后我在 GitHub 上看到了 MemOS。

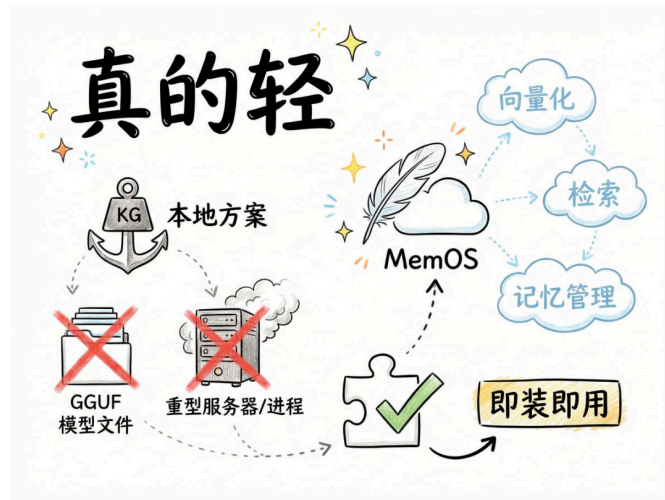
说实话，做 AI 记忆的项目不少，mem0、supermemory、memU 都试过。

但 MemOS 有几个点，让我决定深入试试。

Enhanced OpenClaw with MemOS Plugin

第一，它是真的轻。

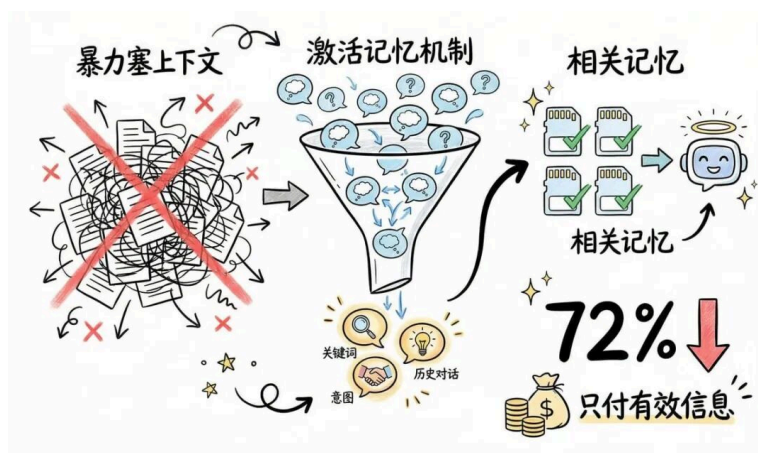
跟本地方案思路完全不同，MemOS 把最吃资源的活，向量化、检索、记忆管理，全部云端化了。不需要下载 GGUF 模型，不需要后台常驻重型进程，只需要一个轻量 Plugin，几行配置，即装即用。



第二，不是暴力塞上下文，而是"精准召回"。

MemOS 引入了激活记忆机制，它不会把所有历史对话一股脑塞给模型，而是基于当前任务意图，精准检索最相关的几条记忆。

只为"有效信息"付费。

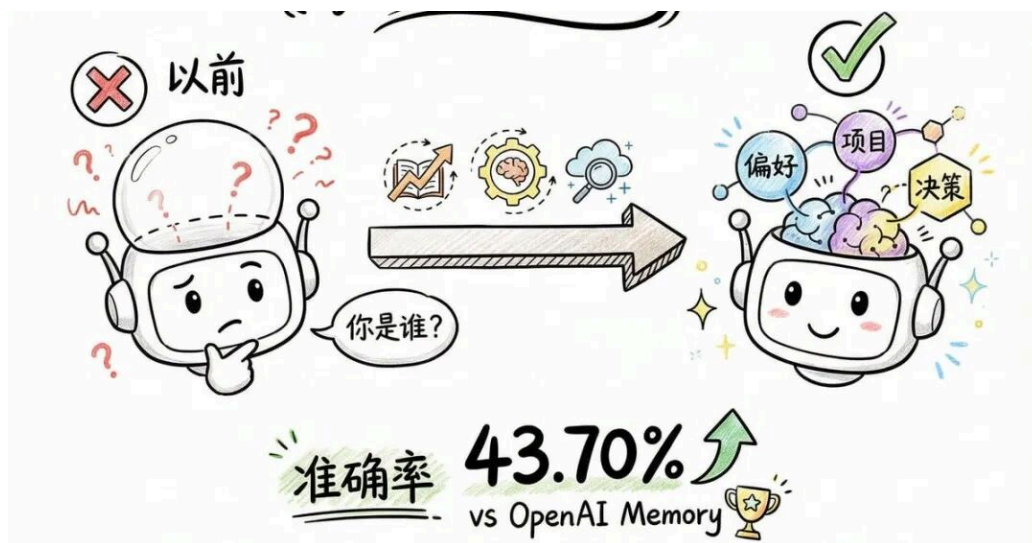


根据官方数据，Token 消耗可以下降72%。我自己体感也很明显，同样的任务，接入前后差距肉眼可见。

第三，个性化体验。

用了一段时间后，你会发现，Agent 真的"认识你"了。

它记得你的偏好，记得你的项目背景，记得你之前的决策。



不像以前，每次开新会话，Agent 就像失忆了一样，你得从头交代一遍。

根据官方评测数据，在 PrefEval-10 基准上，MemOS 准确率比 OpenAI Memory 高出 43.70%。

这不是小数字。

真正让我"回不去"的功能：多 Agent 记忆共享

省钱当然好，但真正让我觉得值回票价的，是 MemOS 解决了多 Agent 协作最大的痛点——Agent之间的记忆隔离。

先说说为什么要用多 Agent。

前面提到了，所有任务塞一个 Agent，时间久了就会错乱。

所以正确做法是，不同任务，不同 Agent，不同工作空间。

我目前常用的几个 Agent：

🧠 头脑风暴Agent：负责发散思维、搜集资料、产出创意，用便宜模型就行，比如 gemini-2.5-flash。

✍️ 公众号写作Agent：负责把想法收敛成文章。

💻 Coding Agent：负责写代码、调试。



这种拆分好处很明显。

每个 Agent 记忆独立，不会因为加载复杂记忆而错乱；上下文不会交叉污染，写代码不会蹦出写文章的记忆；成本也好控制，简单任务用便宜模型，复杂任务上强模型。

但问题来了——

拆分之后，Agent 之间变成了信息孤岛。头脑风暴产出的 20 个创意，写作 Agent 根本不知道。你只能手动复制粘贴，充当"人肉传话筒"。

MemOS 解决的就是这个矛盾：工作空间隔离，但关键信息共享。

接入 MemOS 后，玩法变了。

比如我对头脑风暴 Agent 说："帮我出个智能家居控制系统的方案。"

头脑风暴助手

🔍 📷 👤 📅 ...



OpenClaw 机器人 | “基于飞书的个人 AI 操作系统。ClawdBot 能像真人助理一样处理复杂事务：从整理...

回复 林月半子的AI笔记: 你删除了一条消息

Hey! 我是创意策划师。想聊聊什么想法吗? 无论是产品功能、营销活动还是任何创意项目, 我都可以帮你把想法变成结构化的设计方案 🤖



林月半子的AI笔记 | 公众号: 林月半子的AI笔记

我想做一个智能家居控制系统

2 条回复



OpenClaw 机器人 | “基于飞书的个人 AI 操作系统。ClawdBot 能像真人助理一样处理复杂事务：从整理...

回复 林月半子的AI笔记: 我想做一个智能家居控制系统

好项目! 让我先看看当前的项目状态, 然后我们来细化这个想法。

回复 林月半子的AI笔记: 我想做一个智能家居控制系统

明白了! 现在我们来细化一下想法。
你的这个智能家居控制系统, 主要想用在什么场景?
A. 自己家用 (单户/公寓)
B. 多个出租房/民宿管理
C. 商业空间 (办公室、店铺)
D. 其他场景



林月半子的AI笔记 | 公众号: 林月半子的AI笔记

A

1 条回复



OpenClaw 机器人 | “基于飞书的个人 AI 操作系统。ClawdBot 能像真人助理一样处理复杂事务：从整理...

回复 林月半子的AI笔记: A

好的, 自家用!
你家的规模大概是怎么样?
A. 一室/小户型 (<80平米)
B. 两室/三室 (80-150平米)
C. 大户型/复式 (150-300平米)
D. 别墅/多层 (>300平米)



林月半子的AI笔记 | 公众号: 林月半子的AI笔记

B

发送给 头脑风暴助手

Aa 😊 @ ✂️ + ↩️ ➡️

Agent 正常回复，给出方案细节。



整个对话过程中，你不需要额外操作，对话内容会自动同步到 MemOS（后台日志显示为 addMessage）

事件: addMessage



请求详情

响应详情

状态：

SUCCESS

耗时：212ms

Token 输入：433

日志ID：e54ada7531da9c0916d30f3fc9156246

异步模式：true

用户ID：openclaw-user

会话ID：agent:brainstorm:feishu:group:oc_598146241198039b8d9149ded5fb390b

创建时间：2026-02-12 16:10:18

消息：

```
{
  "role": "user",
  "content": "System: [2026-02-12 16:09:18 GMT+8] Feishu[default] message in group oc_598146241198039b8d9149ded5fb390b: 正确 [Feishu oc_598146241198039b8d9149ded5fb390b:ou_675922880acdfa2a55bf98070b996624 Thu 2026-02-12 16:09 GMT+8] ou_675922880acdfa2a55bf98070b996624: 正确 [message_id: om_x100b57ece412f100c11615c78f0c176]"
},
{
  "role": "assistant",
  "content": "好的，设计验证通过。现在将完整设计写入文档。"
},
{
  "role": "assistant",
  "content": "✅ 设计文档已完成并提交到 git。 **工作专注模式设计方案** - 文档: docs/plans/2026-02-12-smart-home-focus-mode.md - 核心功能: 摄像头自动检测 → 环境调适 → 勿扰模式 → 番茄钟 → 自动结束 - 预算: 550-1060 元（摄像头 + 灯泡 + 窗帘 + 音箱） 准备好开始实施了吗？"
}
]
```

元信息：

```
{
  "source": "openclaw"
```

```
source : openai ,  
"sessionKey":  
"agent:brainstorm:feishu:group:oc_598146241198039b8d9149ded5fb390b",  
"agentId": "brainstorm"  
}
```

然后我切到写作 Agent，直接说：“我之前在brainstorm里聊过一个智能家居方案，根据那个方案，帮我写篇公众号文章。”

Agent自动调用 search_memory，精准捞出刚才的方案。



林月半子的AI笔记 | 公众号：林月半子的AI笔记

我之前通过 brainstorm agent 有过一个智能家居的方案，现在根据这个方案，写一篇公众号文章



2 条回复



OpenClaw 机器人 | “基于飞书的个人 AI 操作系统。ClawdBot 能像真人助理一样处理复杂事务：从整理收件箱...

回复 林月半子的AI笔记：我之前通过 brainstorm agent 有过一个智能家居的方案，现在根据这个方案，写一篇公...
明白了！你是和 brainstorm agent 讨论过智能家居方案。让我帮你找到那些内容。

回复 林月半子的AI笔记：我之前通过 brainstorm agent 有过一个智能家居的方案，现在根据这个方案，写...

找到了！我看到了你之前和 brainstorm agent 讨论过的智能家居方案。让我先整理一下核心内容：

方案亮点：

- 预算友好：入门级 <5000元，普通家庭也能玩
- 本地控制：Home Assistant + Zigbee，断网也能用
- 渐进部署：分阶段升级，不用一次全买
- 实用场景：离家/回家自动开关灯、夜间模式、工作专注模式（番茄钟+灯光+勿扰）

目标读者：想尝试智能家居但预算有限、怕麻烦的普通用户

文章定位：实战分享型，用亲身体会的方式告诉大家“智能家居没那么贵、也没那么难”

现在我写一篇有网感、能转发的公众号文章：

用5000块把家变聪明，我的智能家居踩坑日记

前几天回老家，我妈问我：“你说的智能家居到底好不好用？”

我打开手机App，给她看了我的书房——灯光自动调节、窗帘缓缓关闭、番茄钟倒计时提醒。我妈看完说：“这不就是有钱人的玩意儿吗？”

其实不是。

我用5000块就把80平的房子搞定了，而且大部分钱还是花在灯泡和开关上。

为什么要智能家居？

很多人觉得智能家居就是“装逼”，但我用下来发现，真正好用的是那些省心的小事：

- 出门总是忘了关灯？自动搞定
- 半夜回家摸黑找开关？灯光提前亮起
- 工作时总想看手机？勿扰模式强制专注

这些事情单独看都不重要，但加起来就是——日子更轻松了。

我的方案：少花钱多办事

核心设备（第一阶段 ~1000元）

设备	数量	价格	用途
----	----	----	----

设备	数量	价格	用途
树莓派 4B	1	400-800	大脑（运行Home Assistant）
Zigbee网关	1	200-300	连接所有设备
智能开关	3-4	50-80/个	客厅/卧室灯光控制
门窗传感器	2-3	30-50/个	离家/回家检测
温湿度传感器	1-2	40-60/个	温度联动

技术路线：本地化 > 云端

我选了 Home Assistant，原因很简单：

发送给 公众号写手

Aa 😊 @ ✂️ ⊕ ↶ ➡ | ▾

注意，我没指定标签，也没手动保存。

MemOS 的检索基于语义，你用自然语言描述"之前聊过什么"，它就能找到。

请求详情

响应详情

Token 输出： 2294

状态码： 0

记忆：

```
{
  "id": "1c9a252c-744c-4237-a14d-2d045603cb92",
  "memory_key": "New session initiation",
  "memory_value": "On February 12, 2026, at 3:27 PM, the user initiated a new session in Feishu, which included a system message indicating the start of the session and instructions for greeting the user. The assistant introduced itself as a creative planner, ready to help the user turn ideas into clear design plans.",
  "memory_type": "LongTermMemory",
  "create_time": 1770881304094,
  "conversation_id": "agent:brainstorm:feishu:group:oc_598146241198039b8d9149ded5fb390b",
  "status": "activated",
  "confidence": 0.99,
  "tags": [
    "session",
    "Feishu",
    "introduction",
    "creative planning"
  ],
  "update_time": 1770881304095,
  "relativity": 0.6406329
}
```

说明信息： ok

不需要复制粘贴，不需要重复交代背景。

Agent A 产生的智慧，Agent B 秒懂。

而且整个过程是"无感"的，对话输出会自动回写 MemOS，不需要手动保存，不需要指定格式，它自己完成分类和索引。你要做的，就是正常对话。

同样的逻辑适用于 Coding Agent。

比如我在头脑风暴里讨论完技术方案，切到 Coding Agent 直接说"开始写代码"，它就能从记忆里捞出技术栈和架构，不用再交代一遍。

这才是多 Agent 协作该有的样子。

而且因为每个 Agent 只检索自己需要的记忆，而不是加载全量历史，Token 消耗反而更低了。

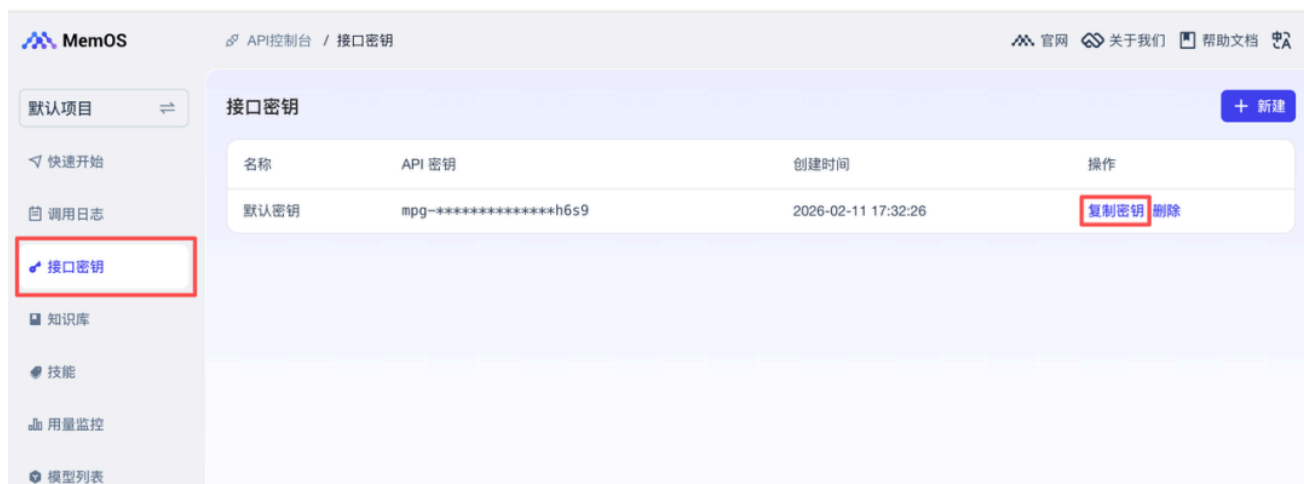
省钱，防错乱，真协作，三件事一次解决。

30秒极速安装

好用的工具必须简单，MemOS 接入流程非常友好。

Step 1：获取API Key

前往MemOS Dashboard (<https://memos-dashboard.openmem.net/cn/login/>) 注册，获取免费 API Key。



Step 2 : 配置环境变量

```
echo "MEMOS_API_KEY=你的key" > ~/.openclaw/.env
```

Step 3 : 一键安装插件

```
openclaw plugins install github:MemTensor/MemOS-Cloud-OpenClaw-Plugin
```

重启后，你的 OpenClaw 就有了一个外置大脑。

写在最后

折腾了这么多 AI 工具，我越来越觉得一件事：

未来AI的竞争，不在于模型参数多大，而在于上下文管理和记忆能力多强。

模型能力在趋同，但记忆管理的差距，决定了你的 Agent 是每次见面都像陌生人，还是一个真正了解你的长期搭档。

MemOS 目前 GitHub上 5.2k Star，Apache 2.0 协议开源，值得关注。

GitHub地址：<https://github.com/MemTensor/MemOS>

觉得有用，顺手去 GitHub 点个 Star，让好项目被更多人看到。

感谢看到这里 🙌

觉得有用的话，点赞 👍 / 在看 👁 / 转发 📤 / 评论 💬

星标 ★ 一下，下次更新不迷路